

ULTRON 2.7 Phase 14 Execution Report

Backend voice capture and offline-first STT pipeline

Project	ULTRON 2.7
Phase	Phase 14: Local Voice Capture and Transcript Safety
Workspace	Local workspace checkout: U2.7
Status	Implemented and verified with mocked provider tests

Executive Summary

Phase 14 replaces browser-only voice capture with a stronger backend voice pipeline. ULTRON can now capture audio through a provider abstraction, analyze audio before transcription, reject empty or noisy input, clean transcripts, track confidence, and expose detailed voice diagnostics to the interface.

The safety chain remains unchanged. Voice input still becomes text first, then flows through the conversation manager, brain/runtime, typed tool call validation, policy gate, executor restrictions, audit logging, and response generation.

Phase 14 Scope

- Add backend microphone capture support through capture providers.
- Keep faster-whisper as the preferred local STT path while preserving whisper.cpp, browser, text payload, and mock fallbacks.
- Add audio duration, energy, speech-duration, noise, and too-short checks before STT.
- Add confidence handling so uncertain transcripts ask for clarification instead of executing.
- Clean transcripts by removing wake words, filler words, repeated words, and command prefixes while preserving command content.
- Expose microphone, provider, transcript, confidence, audio duration, speech duration, and rejection diagnostics.
- Update the web UI with backend capture controls and understood-transcript diagnostics.
- Add mocked tests for capture, cleanup, low confidence, rejected audio, and fallback behavior.

What Was Implemented

Area	File(s)	Result
Configuration	src/ultron27/config.py, ultron.config.example.json	Adds capture provider, microphone device, sample rate, and capture duration options with environment-variable overrides.

Area	File(s)	Result
Voice pipeline	src/ultron27/voice.py	Adds capture provider protocols, browser/mock/sounddevice capture adapters, audio diagnostics, transcript cleanup, confidence checks, and capture-and-run flow.
Local API	src/ultron27/web_server.py	Adds POST /api/voice/capture and returns voice diagnostics with every voice response.
Interface	web/index.html, web/app.js, web/styles.css	Adds Backend Mic controls, active capture provider display, last transcript, confidence, and audio-state panels.
Tests	tests/test_pipeline.py	Adds mocked tests for capture execution, unavailable capture, audio rejection, and cleanup.
Docs	README.md, docs/architecture.md	Documents backend capture setup, diagnostics, and the voice safety boundary.

Voice Safety Flow

Stage	Phase 14 Behavior	Safety Effect
Capture	Audio is collected by a configured provider or safely falls back to browser/mock mode.	Missing local dependencies do not crash the app.
Audio gate	Duration, speech-duration, energy, and noise checks run before STT.	Empty and noisy input is ignored before it can become a command.
Transcription	Provider confidence is tracked and cleaned transcript text is stored in diagnostics.	Low-confidence transcripts ask for clarification.
Runtime	Accepted transcript enters the existing conversation and brain/runtime chain.	Voice never bypasses validation, policy, executor, or audit logging.
UI	The interface shows listening, speech detected, transcribing, understood text, and confidence warnings.	The user can see what ULTRON thinks it heard before action is trusted.

Verification Results

Check	Command or Coverage	Result
Compile check	python -m compileall src/ultron27	Python modules compiled successfully.
Unit tests	python -m unittest discover -s tests	89 tests passed.
Mocked capture	MockMicrophoneCapture through VoiceSession.capture_and_run	Audio payload becomes a transcript and enters the safe runtime.
Noisy/short audio	analyze_audio_payload test coverage	Too-short and noisy audio is rejected safely.
Unavailable capture	Capture provider unavailable path	The API returns a nonfatal unavailable status.
Transcript cleanup	Wake word, filler, and repeated-word cleanup tests	Transcript cleanup preserves command content.

Safety Impact

Phase 14 improves the reliability of the input layer without increasing execution authority. Audio capture and transcription only produce candidate text. The existing safe execution model still decides whether any action can run.

Destructive or high-risk commands remain blocked or confirmation-gated by the runtime and policy system.

Recommended Next Step

- Install and configure a local faster-whisper model for real offline STT testing.
- Add a calibration screen that helps the user choose microphone level and silence thresholds.
- Move next into executor skills for Notepad typing and true Spotify playback control.